

Task 2: Baseline and Proposed Graph Neural Network Model for GRABMyo Gesture Recognition

1. Introduction

Electromyography (EMG) signals provide valuable information about muscle activity and can be used for recognizing different hand gestures. In this task, the GRABMyo dataset was used to develop and evaluate machine learning and graph neural network approaches for multi-class hand gesture recognition.

The complete work was organized into two notebooks:

1. **GroupNN_GRABMyoPC_task2_baselines.ipynb**
2. **GroupNN_GRABMyoPC_task2_proposed_model.ipynb**

The first notebook establishes conventional machine learning baselines using statistical EMG features. The second notebook introduces the proposed Graph Neural Network (GNN) approach by representing the EMG channels as nodes in a graph.

The main objective is to determine whether the proposed graph-based approach can effectively model relationships between EMG channels and provide competitive performance compared with conventional machine learning models.

2. Dataset

The GRABMyo dataset contains EMG recordings collected from multiple participants performing different hand gestures.

For this task, statistical features were extracted from the EMG recordings rather than directly using the raw signals.

The final feature dataset contains:

- **15,351 samples**
- **32 EMG channels**
- **7 statistical features per channel**
- **224 input features**
- **17 gesture classes**
- **43 participants**

The seven statistical features extracted from each channel were:

1. Mean
2. Standard Deviation
3. Root Mean Square (RMS)
4. Maximum
5. Minimum
6. Skewness
7. Kurtosis

Therefore:

32 channels × 7 statistical features = 224 features

In addition to the 224 feature columns, the dataset contains:

- **Gesture** — target class
- **Participant** — participant identifier

Thus, the final dataset contains **226 columns** in total.

Overall Task 2 Workflow

The complete methodology consists of two stages.

Notebook 1 — Baseline Models

The baseline notebook follows:

Load Dataset → Signal Processing → Feature Extraction → Feature Dataset → Subject-wise Split → Feature Scaling → Baseline Models → Evaluation → Comparison

Notebook 2 — Proposed GNN

The proposed notebook follows:

Feature Dataset → Subject-wise Split → Feature Scaling → Graph Construction → Graph Topology Experiment → GCN/GraphSAGE/GAT → Evaluation → Comparison with Baselines

This two-stage design provides a reference point before evaluating the proposed graph-based model.

PART A — BASELINE MODEL EVALUATION

Objective of the Baseline Experiment

The objective of the baseline experiment is to establish the performance of conventional machine learning algorithms on the extracted EMG features.

These results provide a reference against which the proposed GNN can later be compared.

Five machine learning models were evaluated:

1. Decision Tree
2. Random Forest
3. K-Nearest Neighbors (KNN)
4. Support Vector Machine (SVM)
5. Multi-Layer Perceptron (MLP)

Default model parameters were used because these models were intended as baseline models rather than optimized models.

Dataset Loading and Signal Processing

All available GRABMyo recordings were loaded recursively from the dataset directory.

A total of: **15,351 recordings** were processed.

The EMG recordings were processed before statistical feature extraction.

The purpose of signal processing was to prepare the EMG signals for reliable feature extraction.

Feature Extraction

Statistical features were calculated independently for each of the 32 EMG channels.

For every channel, seven features were extracted:

Feature	Description
Mean	Average signal value

Standard Deviation	Signal variability
RMS	Overall signal magnitude
Maximum	Highest signal value
Minimum	Lowest signal value
Skewness	Asymmetry of signal distribution
Kurtosis	Shape/tailedness of signal distribution

This resulted in:

$32 \times 7 = 224$ features per sample

The gesture label and participant identifier were also retained.

Feature Dataset

All extracted features were combined into a pandas DataFrame.

The dataset therefore contains:

Component	Number
EMG feature columns	224
Gesture label	1

Participant ID	1
Total columns	226
Samples	15,351
Gesture classes	17
Participants	43

Before model training, the dataset was checked for:

- Missing values
- Gesture class distribution
- Number of unique participants
- Correct feature dimensions

The dataset was reported as having no missing values, balanced gesture classes, 43 participants, and the expected 224 feature dimensions.

Subject-wise Train-Test Split

A subject-wise split was used to prevent participant information from appearing in both training and testing data.

The splitting method used was: **GroupShuffleSplit**

The split was:

- **80% Training**
- **20% Testing**

This produced:

- **34 participants for training**
- **9 participants for testing**

No participant appears in both sets.

This is important because a random sample-level split could allow samples from the same participant to occur in both training and testing. A participant-wise split instead evaluates whether the model can generalize to unseen participants.

Therefore, the evaluation is **subject-independent** / **participant-independent**.

Feature Scaling

The 224 input features were standardized using: **StandardScaler**

The scaler was fitted only using the training data and subsequently applied to the test data.

This prevents information from the test set from influencing the feature scaling process.

The basic process was:

Training features → Fit scaler → Transform training

and:

Test features → Transform using the training scaler

Baseline Models

Five conventional machine learning models were trained.

Decision Tree

A Decision Tree learns a sequence of decision rules to separate the different gesture classes.

Random Forest

Random Forest combines multiple decision trees to improve robustness and generalization.

K-Nearest Neighbors

KNN classifies a sample based on the classes of nearby training samples.

Support Vector Machine

SVM attempts to find decision boundaries that effectively separate the gesture classes.

Multi-Layer Perceptron

MLP is a feed-forward neural network that learns nonlinear relationships among the 224 statistical features.

Baseline Evaluation Metrics

Each baseline model was evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Training Time

Classification reports and confusion matrices were also generated. For the multi-class metrics, weighted averaging was used.

Baseline Results

The baseline models produced the following accuracy results:

Model	Accuracy
Multi-Layer Perceptron (MLP)	0.6197
Support Vector Machine (SVM)	0.5954
Random Forest	0.5795
K-Nearest Neighbors (KNN)	0.4040

Decision Tree	0.3623
---------------	--------

The MLP achieved the highest baseline accuracy of: **61.97%**

Its reported metrics were:

Metric	MLP
Accuracy	61.97%
Precision	63.62%
Recall	61.97%
F1-score	62.08%

13. Best Baseline Model

Among the five baseline models, the **Multi-Layer Perceptron (MLP)** achieved the highest performance.

Therefore, MLP is used as the main conventional machine learning reference when discussing the performance of the proposed GNN approach.

The baseline experiment demonstrates that the extracted statistical EMG features contain substantial information for gesture recognition.

PART B — PROPOSED GNN MODEL

Motivation for the Proposed GNN

Conventional machine learning models treat the 224 extracted features as a flat feature vector.

However, the features originate from different EMG channels.

Therefore, instead of considering all 224 values independently, the proposed approach represents the EMG channels as nodes in a graph.

The graph representation allows the GNN to learn information from relationships between connected EMG channels.

The proposed approach therefore investigates whether graph-based representation can improve the modeling of EMG channel relationships.

Converting EMG Features into a Graph

The 224 features are organized as:

32 EMG channels $\times$ 7 statistical features

Therefore:

$$224 = 32 \times 7$$

Each sample is reshaped from:

224 features

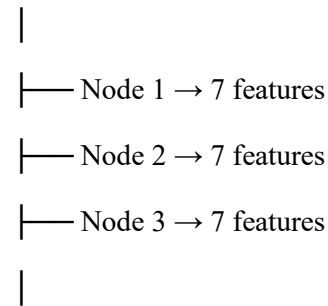
into:

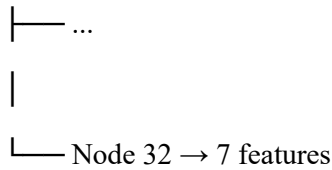
$$32 \times 7$$

The 32 channels become graph nodes, while the seven statistical values become the feature vector associated with each node.

Thus, one sample can be represented as:

Graph





This gives the GNN:

32 nodes × 7 features per node

for each graph sample.

Input, Target and Participant Information

The input features are represented by: X

The target gesture labels are represented by: y

The participant identifiers are represented separately as: $groups$

The participant information is not provided to the classifier.

It is only used to ensure participant-wise dataset splitting.

The model predicts one of the: **17 gesture classes**

Train, Validation and Test Sets

For the proposed GNN experiment, three sets were used:

Training Set

Used to learn model parameters.

Validation Set

Used to select the best graph topology, model configuration, and best training state.

Test Set

Used only for the final evaluation.

The workflow is therefore:

Training → Learn

Validation → Select

Testing → Final Evaluation

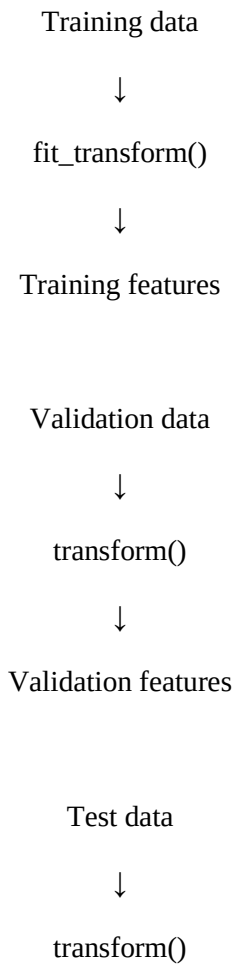
This prevents the test set from being used during model selection.

Feature Scaling for GNN

The features were standardized using `StandardScaler`.

The scaler was fitted only on the training features and then applied to validation and test features.

The process was:





Test features

This avoids information leakage from validation and test data.

Graph Construction

Nodes alone do not define a graph.

The GNN also requires information about which nodes are connected.

This connectivity is represented using:

`edge_index`

For example:

Node 1 — Node 2 — Node 3

means that the graph contains connections between these nodes. The graph structure determines how information can propagate between EMG channels.

Graph Topology Experiment

Rather than manually selecting one graph structure, several graph topologies were experimentally evaluated.

The following five graph structures were compared:

1. Chain
2. Ring
3. Skip
4. Chain + Skip
5. Fully Connected

The same GCN configuration was used for the topology experiment, while only the graph structure was changed. This creates a fair comparison of graph structures.

Chain Graph

The Chain topology connects neighboring nodes:

1 — 2 — 3 — 4 — ... — 32

For 32 nodes, the graph contains: **62 directed edges**

because the connections are represented in both directions.

Ring Graph

The Ring topology connects neighboring nodes and additionally connects the final node back to the first node:

1 — 2 — 3 — ... — 31 — 32



For 32 nodes, it contains:

64 directed edges

23. Skip Graph

The Skip topology allows information to move across one intermediate node.

For example:

1 → 3

2 → 4

$3 \rightarrow 5$

...

For 32 nodes, it contains: **60 directed edges**

This provides longer-range connections than a simple chain.

Chain + Skip Graph

The Chain + Skip topology combines:

- Normal neighboring connections
- Skip connections

This allows information to propagate both locally and across one additional step.

It contains: **122 directed edges**

Fully Connected Graph

In the Fully Connected topology, every node is connected to every other node.

For 32 nodes: $32 \times 31 = 992$ **directed edges**

This is substantially denser than the other graph structures.

Graph Topology Selection

Each topology was trained using the same GCN configuration to find out which **graph structure provides the most useful representation of relationships between EMG channels**.

The graph topology results from the previous experiment were:

Graph Topology	Validation Accuracy
----------------	---------------------

Ring	0.2673
Chain	0.2669
Skip	0.2641
Chain + Skip	0.2561
Fully Connected	0.2305

The **Ring topology** achieved the **highest validation accuracy in that experiment**.

However, Ring and Chain were very close, so the result should be described as the best-performing topology **among the evaluated structures**, rather than as proof that Ring is universally optimal.

The selected graph topology was then used for comparing the GCN, GraphSAGE, and GAT architectures.

Proposed GNN Architectures

Three GNN architectures were evaluated:

1. GCN
2. GraphSAGE
3. GAT

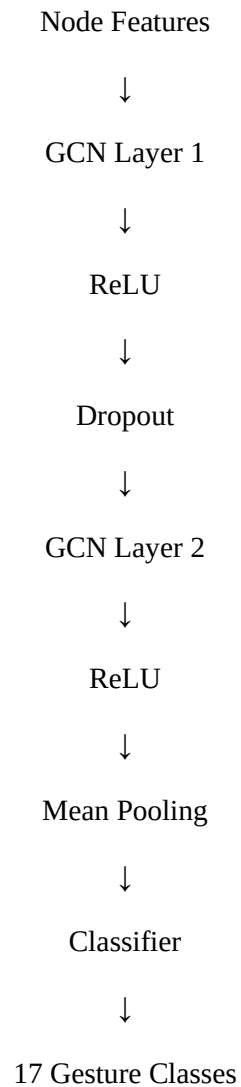
The same selected graph topology was used for the three architectures to make the model comparison fair.

The purpose is to determine whether differences in performance arise from the GNN architecture rather than from using different graph structures.

GCN

The Graph Convolutional Network uses neighboring node information to generate updated node representations.

The architecture consists conceptually of:



The configuration used included:

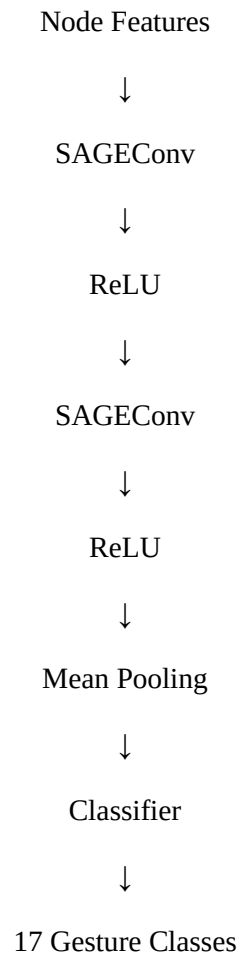
- Input features per node: **7**
- Hidden layer 1: **32**
- Hidden layer 2: **64**
- Pooling: **Mean**
- Output classes: **17**

GraphSAGE

GraphSAGE stands for **Graph Sample and Aggregate**.

Instead of using the exact aggregation mechanism of GCN, GraphSAGE learns node representations by aggregating information from neighboring nodes.

Its general flow is:



The purpose of including GraphSAGE is to compare a neighbor-aggregation approach against the GCN architecture.

GAT

GAT stands for **Graph Attention Network**.

GAT uses attention mechanisms to learn that different neighboring nodes may have different levels of importance.

Conceptually:

Neighbor A → High importance

Neighbor B → Low importance

Neighbor C → Medium importance

The implemented GAT used:

4 attention heads

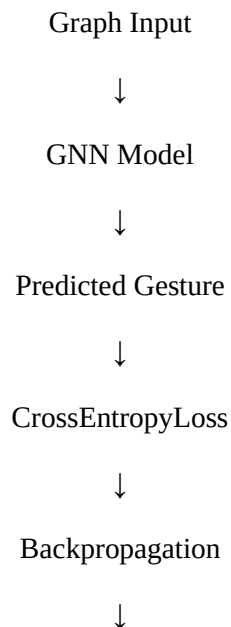
The attention-based architecture allows the model to learn which neighboring EMG channels are more informative for gesture recognition.

Training Procedure

The GNN models were trained using: **CrossEntropyLoss** because the task contains 17 gesture classes.

The optimization method was: **Adam optimizer**

The training procedure is:



Adam Optimizer



Update Parameters

The model was trained for up to **30 epochs** during the graph topology experiment.

The best validation model state was saved rather than automatically using the final epoch.

GNN Evaluation Metrics

The final GNN models were evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Training Time

The test set was kept separate from the model-selection process.

Proposed GNN Results

The latest proposed-model experiment produced the following results:

Model	Validation Accuracy	Test Accuracy	Precision	Recall	F1-score	Training Time (s)
GAT	0.3537	0.3041	0.3619	0.3041	0.2933	79.4229
GraphSAGE	0.3557	0.3019	0.3381	0.3019	0.2943	50.1274
GCN	0.3537	0.2487	0.2434	0.2487	0.2273	59.0397

Analysis of GNN Results

GAT

GAT achieved the highest test accuracy: **30.41%**. It also achieved the highest test precision: **36.19%**

This indicates that the attention mechanism was useful for identifying important relationships among neighboring EMG channels.

GraphSAGE

GraphSAGE achieved the highest validation accuracy: **35.57%** and the highest F1-score: **29.43%**

It also had the shortest training time: **50.13 seconds** among the three GNN models.

GCN

GCN achieved: **24.87% test accuracy** which was lower than both GraphSAGE and GAT.

Therefore, under the evaluated configuration, the more advanced aggregation/attention approaches performed better than the basic GCN.

Baseline vs Proposed Model

The best conventional baseline was MLP: **61.97% test accuracy**

The best proposed GNN test accuracy was obtained by GAT: **30.41%**

Therefore, based on the current experiment, the conventional MLP baseline performs substantially better than the proposed GNN.

The comparison:

Approach	Model	Test Accuracy
Baseline	MLP	61.97%

Baseline	SVM	59.54%
Baseline	Random Forest	57.95%
Baseline	KNN	40.40%
Baseline	Decision Tree	36.23%
Proposed GNN	GAT	30.41%
Proposed GNN	GraphSAGE	30.19%
Proposed GNN	GCN	24.87%

Conclusion

This task developed and evaluated two different approaches for GRABMyo hand gesture recognition.

First, several conventional machine learning algorithms were trained using 224 statistical EMG features. Among the five baseline models, the **Multi-Layer Perceptron achieved the highest performance**, with an accuracy of **61.97%**, precision of **63.62%**, recall of **61.97%**, and F1-score of **62.08%**.

Second, a graph-based representation was developed by transforming the 224 statistical features into **32 graph nodes**, where each node represents an EMG channel and contains seven statistical features. Multiple graph topologies were experimentally evaluated before comparing three GNN architectures: GCN, GraphSAGE, and GAT.

Among the evaluated GNN models, **GAT achieved the highest test accuracy of 30.41%**, while GraphSAGE achieved the highest validation accuracy of 35.57% and the highest F1-score of 29.43%.

However, the current GNN models did not outperform the MLP baseline. The MLP achieved 61.97% test accuracy compared with 30.41% for the best GNN.

Therefore, the current results suggest that the extracted statistical features can be effectively used by conventional neural models, while the proposed graph representation requires further improvement. Future work should investigate data-driven graph construction, GNN hyperparameter optimization, alternative pooling and attention configurations, and more robust multi-run evaluation.

-----*****-----